ENGENHARIA DE SOFTWARE: ROTEIRO DE PRÁTICA DE LABORATÓRIO

ENTREGA 2 DE 3: Liberação de Origem Explícita e Indução de Preflight (OPTIONS)

Autorização de Requisições Simples com `Access-Control-Allow-Origin` e Quebra de Simplicidade via Cabeçalho Customizado

---

Informações Gerais	Detalhes
Janela de Execução Estimada	40 minutos (Bloco 2)
Stack Tecnológica	JavaScript Nativo Standard Runtime + Fetch API do Navegador
Requisitos de Instalação Local	**ZERO INSTALAÇÃO** (Execução 100% via Navegador Web)
Peso na Avaliação	0,25 pontos na Nota do Bimestre

---

1. Objetivo Pedagógico e Fundamentação Teórica

Na **Entrega 1**, comprovamos que o navegador bloqueia e descarta a leitura de respostas vindas de origens diferentes (*Cross-Origin*) quando o servidor não declara expressamente quem tem permissão para consumi-las. Para liberar a leitura dessa resposta, o servidor deve anexar o cabeçalho HTTP:

Access-Control-Allow-Origin: ⟨origem-confiável⟩

Contudo, a especificação do W3C / WHATWG divide as chamadas HTTP *Cross-Origin* em duas categorias cruciais:

1. **Requisições Simples (*Simple Requests*):**
    - Utilizam exclusivamente métodos seguros (`GET`, `HEAD` ou `POST`).
    - Utilizam apenas cabeçalhos normatizados pelo navegador (`Accept`, `Accept-Language`, `Content-Language` ou `Content-Type` restrito a `text/plain`, `multipart/form-data` ou `application/x-www-form-urlencoded`).

- **Comportamento:** O navegador dispara a requisição diretamente ao servidor e só checa o cabeçalho `Access-Control-Allow-Origin` quando a resposta retorna.

2. **Requisições com Pré-verificação (*Preflighted Requests*):**
    - Ocorrem quando a requisição utiliza métodos como `PUT`, `DELETE`, `PATCH`, transporta payloads JSON no corpo, ou anexa **cabeçalhos HTTP proprietários/customizados** (por exemplo, `X-Identificador-Estudante`, `Authorization`, `X-Api-Key`).
    - **Comportamento:** O navegador **NÃO** envia os dados imediatamente. Antes, ele dispara de forma autônoma uma requisição prévia de sondagem com o método HTTP `OPTIONS` (*Preflight*) para interrogar o servidor se aquele cabeçalho e método são permitidos.

O QUE VOCÊ FARÁ NESTA ETAPA 2

Fase A (Liberação Simples):

Configurar 'Access-Control-Allow-Origin' com a origem exata do seu cliente no servidor da API, validando que a resposta JSON passa a ser recebida e renderizada com sucesso no navegador.

Fase B (Indução de Preflight):

Modificar a chamada fetch() no cliente para anexar o cabeçalho customizado 'X-Identificador-Estudante'. Isso forçará o navegador a emitir uma sondagem OPTIONS, que receberá status de erro da API atual (comprovando na prática a quebra de simplicidade).

2. Preparação do Ambiente "Zero-Install"

2.1. Execução em Nuvem via Navegador (Recomendada)

Esta opção utiliza o **GitHub Codespaces**. Não é necessário baixar nada no computador físico; todo o ambiente roda no próprio navegador (Google Chrome, Edge ou Firefox).

1. Abra o navegador da máquina e acesse https://github.com/codespaces (ou acerte o login no GitHub).
2. Clique em **"Blank" / "New codespace"** para abrir um terminal e editor VS Code completo rodando na nuvem.
3. Crie a estrutura de pastas do projeto:
    - Pasta `api/` contendo o arquivo `server-api.js`
    - Pasta `client/` contendo o arquivo `index.html` e `server-client.js`
4. Quando rodar os servidores, o ambiente gerará URLs seguras para cada porta (ex.: `https://seu-espaco-3000.app.github.dev` e `https://seu-espaco-8080.app.github.dev`). Ambas são origens distintas, permitindo testar o CORS perfeitamente.

> **Nota para Portas no Codespaces:** Na aba **Ports** do terminal inferior, certifique-se de que a visibilidade da porta 8080 e da porta 3000 esteja configurada como **Public** (botão direito na porta → *Port Visibility* → *Public*), permitindo que o navegador realize as requisições sem interceptação de tela de login.

3. Estado Inicial dos Arquivos (Ponto de Partida da Etapa 2)

Certifique-se de que os arquivos do seu laboratório estejam organizados conforme a estrutura a seguir:

```
meu-projeto-cors/
├── api/
│   └── server-api.js       <-- Servidor HTTP da API (Porta 8080)
└── client/
    ├── index.html          <-- Interface do usuário (Porta 3000)
    └── server-client.js    <-- Servidor estático simples para o frontend (Porta 3000)
```

Arquivo 1: `api/server-api.js` (Base construída na Entrega 1)

```
const http = require('http');
const PORT = 8080;

http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/dados') {
    res.writeHead(200, {
      'Content-Type': 'application/json; charset=utf-8'
    });
    res.end(JSON.stringify({
      matricula: 'SUA_MATRICULA_REAL',
      status: 'dados_sensiveis'
    }));
    return;
  }

  res.writeHead(404, { 'Content-Type': 'application/json; charset=utf-
8' });
  res.end(JSON.stringify({ erro: 'Nao encontrado' }));
}).listen(PORT, () => {
  console.log(`[API] Servidor ativo na porta ${PORT}`);
});
```

Arquivo 2: `client/server-client.js` (Servidor Nativo do Frontend)

Este arquivo serve o `index.html` na porta 3000 sem precisar de pacotes externos como `live-server` ou `http-server`.

```
const http = require("http");
const fs = require("fs");
const path = require("path");
const PORT = 3000;

http
  .createServer((req, res) => {
    const filePath = path.join(__dirname, "index.html");
    fs.readFile(filePath, (err, content) => {
      if (err) {
        res.writeHead(500, { "Content-Type": "text/plain; charset=utf-
8" });
        res.end("Erro ao carregar index.html");
        return;
      }
      res.writeHead(200, { "Content-Type": "text/html; charset=utf-8"
});
      res.end(content);
    });
  })
  .listen(PORT, () => {
    console.log(`[CLIENTE] Frontend ativo na porta ${PORT}`);
  });
```

Arquivo 3: `client/index.html` (Interface do Aluno)

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Prática CORS - Laboratório</title>
  <style>
    body { font-family: sans-serif; margin: 2rem; background: #f4f6f8;
}
    .card { background: #fff; padding: 1.5rem; border-radius: 8px;
box-shadow: 0 2px 4px rgba(0,0,0,0.1); max-width: 600px; }
    button { padding: 10px 16px; background: #0066cc; color: #fff;
border: none; border-radius: 4px; cursor: pointer; font-size: 14px; }
    button:hover { background: #004c99; }
    pre { background: #1e1e1e; color: #d4d4d4; padding: 1rem; border-
radius: 4px; overflow-x: auto; }
```

```
    .erro { color: #d93025; font-weight: bold; }
  </style>
</head>
<body>
  <div class="card">
    <h2>Painel do Cliente (Frontend)</h2>
    <p>Origem do Cliente: <strong id="origem-
atual">Detectando...</strong></p>
    <button id="btn-requisicao">Executar Requisição</button>
    <h3>Resultado:</h3>
    <pre id="saida">Nenhuma requisição disparada ainda.</pre>
  </div>

  <script>
    document.getElementById('origem-atual').innerText =
window.location.origin;

    // DEFINA A URL DA API:
    // Se estiver em localhost: 'http://localhost:8080/dados'
    // Se estiver no Codespaces: use a URL completa da porta 8080 (ex:
'https://xxx-8080.app.github.dev/dados')
    const API_URL = 'http://localhost:8080/dados';

    document.getElementById('btn-
requisicao').addEventListener('click', () => {
      const saida = document.getElementById('saida');
      saida.innerText = 'Disparando requisição...';
      saida.classList.remove('erro');

      fetch(API_URL)
        .then(res => {
          if (!res.ok) throw new Error(`HTTP Error: ${res.status}`);
          return res.json();
        })
        .then(dados => {
          saida.innerText = JSON.stringify(dados, null, 2);
        })
        .catch(err => {
          saida.classList.add('erro');
          saida.innerText = 'Falha na requisição (CORS ou Conexão):\n'
+ err.message;
        });
    });
  </script>
</body>
</html>
```

4. Exemplos Análogos Prévios: Respostas com Múltiplos Cabeçalhos e Requisições Customizadas

Antes de aplicar as alterações da Etapa 2, analise os dois padrões sintáticos a seguir:

EXEMPLO A — Enviando Múltiplos Cabeçalhos de Resposta no Servidor

```
// Dentro do callback de tratamento do servidor HTTP:
if (req.method === 'GET' && req.url === '/relatorio-financeiro') {
// O objeto no segundo argumento de writeHead recebe múltiplos pares
de chave/valor:
res.writeHead(200, {
'Content-Type': 'application/json; charset=utf-8',
'Access-Control-Allow-Origin': 'http://painel-contabil.local:4200', //
Origem explicitamente autorizada
'X-Versao-API': '1.4.2'
});
res.end(JSON.stringify({ saldo: 15400.50, moeda: 'BRL' }));
return;
}
```

EXEMPLO B — Passando Objeto de Configuração e Headers Customizados no `fetch()`

```
// No JavaScript do cliente (navegador):
// O segundo argumento de fetch() recebe um objeto de configuração
(RequestInit):
fetch("http://localhost:9000/relatorio-financeiro", {
  method: "GET",
  headers: {
    Accept: "application/json",
    "X-Codigo-Filial": "filial-curitiba-01", // Cabeçalho proprietário
não-padrão (provoca Preflight OPTIONS!)
  },
})
  .then((res) => {
    if (!res.ok) throw new Error(`HTTP Error: ${res.status}`);
    return res.json();
  })
  .then((data) => console.log("Sucesso:", data))
  .catch((err) => console.error("Falha:", err));
```

5. Roteiro Detalhado de Execução — Passo a Passo

Siga rigorosamente cada etapa. **Não pule a validação intermediária (Passo 2)**, pois ela comprova que a autorização de origem simples funciona antes de induzirmos a falha de preflight.

Passo	Ação / Etapa	Detalhamento Técnico e Procedimento
Passo 1	**Declarar a Origem Permitida na API**	Abra `api/server-api.js`. Na rota `GET /dados`, edite o `res.writeHead(200, { ... })` e adicione o cabeçalho `'Access-Control-Allow-Origin'`. • Se estiver usando **localhost**, atribua `'http://localhost:3000'`. • Se estiver usando **Codespaces**, atribua a URL exata do frontend (ex.: `'https://seu-id-3000.app.github.dev'`). *(Atenção: É terminantemente proibido o uso de curinga `'*'`).*
Passo 2	**Reiniciar a API e Validar a Liberação Simples**	No terminal onde roda a API, pare a execução com `Ctrl + C` e reinicie: `node api/server-api.js`. No navegador, acerte a aba do cliente e clique em **"Executar Requisição"**. **Validação Esperada:** O JSON contendo sua matrícula real deve ser renderizado na tela e **nenhum erro** deve constar no Console do DevTools.
Passo 3	**Inserir Cabeçalho Customizado no Cliente**	Abra `client/index.html`. Localize a instrução `fetch(...)` e altere-a para incluir o segundo argumento com `method: 'GET'` e o cabeçalho proprietário `X-Identificador-Estudante` preenchido com a sua matrícula (conforme demonstrado no Exemplo Didático B).
Passo 4	**Habilitar Coluna "Method" no DevTools**	Salve o arquivo `client/index.html`. No navegador, pressione `F5` (ou `Ctrl + R` / `Cmd + R`) para atualizar a página. Abra o DevTools (`F12`), clique na aba **Network (Rede)**. Caso a coluna **Method** não esteja visível na lista de requisições, clique com o botão direito no cabeçalho das colunas (*Name*, *Status*, etc.) e ative a opção **Method**.
Passo 5	**Disparar Requisição e Inspecionar o Preflight**	Limpe o histórico do DevTools (ícone 🚫) e clique no botão **"Executar Requisição"**. **Validação Esperada:** A página voltará a exibir mensagem de erro! Observe na tabela do Network que o navegador **NÃO** enviou um `GET`, mas sim uma requisição automática com Method =

Passo	Ação / Etapa	Detalhamento Técnico e Procedimento
		OPTIONS para `/dados`, recebendo status **404 Not Found** (ou erro de CORS). No Console, leia a mensagem de erro acusando a falha no pré-voo (*preflight request*).

6. Especificação Rigorosa da Entrega 2

Submeta no Ambiente Virtual de Aprendizagem (AVA) um único arquivo compactado nomeado como `Entrega2_NUM-RA.zip` contendo os artefatos descritos na tabela abaixo:

Item	Artefato Exigido	Critérios Rigorosos de Aceite
1	`api/server-api.js`	Arquivo da API contendo:• O cabeçalho `'Access-Control-Allow-Origin'` preenchido explicitamente com a origem do cliente (sem uso do curinga `'*'`) dentro do bloco da rota `GET /dados`.
2	`client/index.html`	Arquivo do frontend contendo:• A função `fetch()` configurada com o objeto de opções passando `method: 'GET'` e o cabeçalho customizado `'X-Identificador-Estudante'` com o número de matrícula real do estudante.
3	`print-entrega2.png`	Captura de tela nítida do navegador demonstrando a realização do **Passo 5** com o DevTools aberto: • **Aba Network:** Coluna *Method* visível exibindo com clareza o método **OPTIONS** disparado para `/dados` com retorno de erro; • **Aba Console:** Mensagem de erro visível indicando falha de Preflight (ex.: *"Response to preflight request doesn't pass access control check..."*).